

Database

Mattia Masina

CdL in Informatica – Università di Bologna

Anno Accademico 2026/2027

Indice

1	Basi di Dati e Sistemi di Gestione dei Dati (DBMS)	3
1.1	Metodologia di Studio e Sistemi Informativi	3
1.2	Dati, Informazione e Codifica	5
1.3	Basi di Dati, DBMS e Confronto con i File System	6
1.4	Modelli dei Dati, Architettura e Indipendenza	10
1.5	Linguaggi per Basi di Dati e Modalità di Utilizzo	12
1.6	Attori, Ruoli, Transazioni e Valutazione dei DBMS	14

1 Basi di Dati e Sistemi di Gestione dei Dati (DBMS)

1.1 Metodologia di Studio e Sistemi Informativi

Studio Individuale e Concetti Fondamentali. Per imparare a gestire le basi di dati non basta mandare a memoria definizioni astratte. È fondamentale sviluppare una forma mentale rigorosa: capire come rappresentare la realtà in modo formale e collegare sempre la teoria a problemi pratici e casi d'uso concreti.

Attività Applicativa e Strumenti Software. La teoria deve andare di pari passo con la pratica. Esercizi, laboratori e progetti d'esame richiedono l'uso operativo di sistemi software dedicati, detti **DBMS** (*Database Management System*). Nel mondo reale e sul mercato esistono diverse categorie di DBMS:

- **Sistemi Enterprise e Commerciali:** IBM DB2, Oracle Database, Microsoft SQL Server (pensati per grandi aziende con carichi elevati).
- **Sistemi Open Source Relazionali Standard:** PostgreSQL, MySQL (molto diffusi nel web e nei progetti moderni).
- **Sistemi Desktop ed Embedded:** Microsoft Access, SQLite (leggeri, spesso integrati dentro singole applicazioni o app per smartphone).
- **Piattaforme Cloud e Analitiche:** Google BigQuery (ottimizzate per analizzare moli immense di dati).

Punto di Vista Metodologico vs Tecnologico. La materia viene affrontata da due angolazioni complementari:

- **Punto di vista metodologico:** riguarda la *progettazione*. Ci si concentra sui modelli logici e concettuali usati per descrivere le informazioni del mondo reale in modo pulito, astruendo dai dettagli fisici del computer.
- **Punto di vista tecnologico:** riguarda il *software*. Esamina come è fatto internamente un DBMS: come scrive i dati sui dischi, come gestisce la memoria, come gestisce accessi simultanei e quali linguaggi offre.

I Quattro Contenuti Fondamentali della Disciplina

- **Modelli per l'organizzazione dei dati:** insiemi di regole e strutture matematiche per descrivere entità, relazioni e vincoli del mondo reale.
- **Linguaggi per l'utilizzo dei dati:** strumenti (come SQL) per inserire, leggere, aggiornare ed eliminare i record.
- **Sistemi per la gestione dei dati:** l'architettura dei DBMS, capace di garantire sicurezza, persistenza ed efficienza.
- **Metodologie di progettazione:** i passi guidati per passare dai requisiti parlati a uno schema di database corretto, privo di errori e facile da mantenere.

Il Sistema Informativo (SI). Il **sistema informativo** è l'insieme di persone, strumenti e procedure con cui un'organizzazione (azienda, università, ospedale) raccoglie, organizza e scambia le informazioni indispensabili per svolgere le proprie attività. Anche se un'azienda non ha un reparto chiamato formalmente "Ufficio Sistema Informativo", possiede comunque un sistema informativo che ne supporta il lavoro quotidiano.

Indipendenza dall'Automazione Informatica. Un sistema informativo **non deve essere per forza computerizzato**.

Esempio: Gli archivi notarili o gli uffici anagrafe comunali dell'Ottocento funzionavano benissimo usando faldoni cartacei, registri scritti a mano e fattorini; costituivano a tutti gli effetti un sistema informativo, pur in totale assenza di computer.

Il Ciclo di Gestione dell'Informazione. Qualsiasi informazione all'interno di un'organizzazione attraversa un ciclo di quattro passaggi:

- **Raccolta e Acquisizione:** reperire i dati (es. far compilare un modulo di iscrizione a uno studente).
- **Archiviazione e Conservazione:** salvare i dati in modo duraturo per poterli ritrovare in futuro (es. registrarli in un archivio o sul disco).
- **Elaborazione e Trasformazione:** calcolare nuovi risultati a partire dai dati grezzi (es. calcolare la media dei voti o il bilancio annuale).
- **Distribuzione e Comunicazione:** trasmettere i risultati a chi ne ha bisogno (es. inviare una ricevuta via email o mostrare una schermata all'operatore).

Il Sistema Informatico (IT System). Il **sistema informatico** è la parte tecnologica e automatizzata del sistema informativo. È composto dall'hardware (server, dischi, reti), dal software (DBMS, programmi applicativi) e dagli specialisti informatici che li gestiscono.

Gerarchia dei Sistemi Aziendali: Modello a Cerchi Concentrici

Dal contesto più ampio a quello più specifico, l'organizzazione si struttura in quattro livelli concentrici:

1. **Impresa (*Enterprise*):** l'ente economico complessivo nel suo mercato (obiettivi di business, clienti, risorse).
2. **Organizzazione (*Organization*):** l'insieme coordinato di persone, uffici, compiti e processi decisionali.
3. **Sistema Informativo (*Information System*):** la parte dell'organizzazione dedicata specificamente alla gestione dei flussi informativi.
4. **Sistema Informatico (*IT System*):** l'infrastruttura tecnologica digitale (computer e software) che automatizza il sistema informativo.

Relazione di inclusione: Enterprise $\supset$ Organization $\supset$ Information System $\supset$ IT System.

Tutti gli informatici vorrebbero che il sistema informativo e quello informatico fossero perfettamente allineati. Il problema è che in certi casi non conviene o non ha senso informatizzare alcune informazioni.

1.2 Dati, Informazione e Codifica

Distinzione Concettuale e Necessità di Interpretazione. Nella vita di tutti i giorni esprimiamo concetti in molti modi: parole parlate, testo scritto, disegni, numeri o gesti. Tali informazioni possono essere salvate nella nostra memoria biologica (il cervello), su carta oppure su supporti digitali. Nei calcolatori, però, tutta questa varietà deve essere ridotta e formalizzata sotto forma di **dati**.

I due termini hanno ruoli distinti:

- **Informazione:** la conoscenza o il significato che una persona ricava da una notizia, da un fatto o da un contesto.
- **Dato:** l'elemento grezzo, preciso e privo di ambiguità (es. una stringa o un numero), memorizzato come punto di partenza per il calcolo.

Il Dato è Inutile senza Interpretazione: L'Esempio dei Segnali Finlandesi

Si considerino tre cartelli stradali finlandesi di divieto di sosta con pannelli integrativi numerici:

Dato Grezzo	Significato Reale (Informazione Compresa)
8 – 17	Divieto attivo dalle 8:00 alle 17:00 nei giorni feriali (Lunedì–Venerdì).
(8 – 13)	Divieto attivo dalle 8:00 alle 13:00 nella giornata di Sabato (indicato tra parentesi).
8 – 14	Divieto attivo dalle 8:00 alle 14:00 nei giorni festivi e Domenica (indicato in rosso).

Analisi Semantica:

I numeri da soli sono solo cifre. Senza conoscere il codice stradale finlandese (il contesto), non sappiamo a cosa si riferiscano parentesi o colori.

⇒ **Regola chiave:** Dato + Contesto / Chiave Interpretativa = Informazione.

Il Processo di Codifica dell'Informazione. Per consentire a un computer di elaborare la realtà, dobbiamo codificare le informazioni secondo regole fisse e ripetibili:

- Si prendono descrizioni del mondo reale ricche e discorsive (es. dati di un cittadino).
- Si scompone il testo in attributi precisi: *Nome, Cognome, Data di Nascita*.
- Si crea un identificatore standard sintetico e univoco: ad esempio il **Codice Fiscale**.

Perché i Dati Sono una Risorsa Strategica. All'interno di qualsiasi ente o azienda, quasi tutto cambia in fretta:

- I processi di business e le strategie commerciali si rinnovano continuamente.
- I computer, i server e i linguaggi di programmazione invecchiano e vengono rimpiazzati in pochi anni.
- Le persone cambiano ruolo o vanno in pensione.
- **I dati invece restano stabili nel tempo:** un contratto, uno storico di bonifici bancari o un registro delle nascite mantengono la propria validità e struttura invariate per decenni. I dati sono quindi il patrimonio più durevole dell'azienda.

1.3 Basi di Dati, DBMS e Confronto con i File System

Definizione di Base di Dati. L'espressione **base di dati** (*database*) viene usata con due significati:

- **Significato Generico:** qualsiasi archivio strutturato di dati usato per supportare le attività di un'organizzazione o di un individuo (anche una rubrica telefonica cartacea).
- **Significato Informatico:** una collezione organizzata di dati, salvata su memorie di massa persistenti (dischi SSD, hard disk) e gestita in modo centralizzato da un programma dedicato chiamato **DBMS**.

Il DBMS (DataBase Management System). Un **DBMS** è un sistema software concepito per gestire grandi collezioni di dati, garantendo che le operazioni siano sicure, affidabili ed efficienti.

Le Tre Proprietà Intrinseche dei Dati Gestiti da un DBMS

Un database gestito da un DBMS possiede tre proprietà distintive:

1. **Grandi Dimensioni (*Big*):** i dati non possono risiedere per intero nella memoria RAM del computer; il sistema sfrutta la memoria secondaria (dischi fisici o storage cloud).
2. **Persistenza (*Persistent*):** i dati sopravvivono alla chiusura delle applicazioni e allo spegnimento della macchina.
3. **Condivisione (*Shared*):** la stessa base di dati è usata contemporaneamente da programmi diversi e da utenti diversi, evitando di dover duplicare gli archivi per ogni ufficio.

I Quattro Requisiti di Garanzia del DBMS. Un DBMS affidabile offre quattro garanzie fondamentali:

- **Riservatezza (*Privacy / Sicurezza*):** ciascun utente vede e modifica solo i dati per cui ha un permesso esplicito (es. uno studente vede il proprio libretto, ma non quello degli altri).

- **Affidabilità (*Reliability*):** in caso di guasto hardware o interruzione improvvisa della corrente elettrica, i dati memorizzati non si corrompono né vanno persi.
- **Efficienza (*Efficiency*):** il sistema risponde velocemente alle richieste senza sprecare memoria o tempo di CPU, sfruttando algoritmi di indicizzazione mirati.
- **Efficacia (*Effectiveness*):** mette a disposizione strumenti intuitivi, potenti e flessibili per estrarre e lavorare sui dati utili al lavoro quotidiano.

Disponibilità dei DBMS sul Mercato. I DBMS si possono installare sui propri server (*on-premise*) oppure noleggiare come servizi gestiti nel cloud (*Database as a Service*). Alcuni dei prodotti più usati sono Oracle, Microsoft SQL Server, PostgreSQL, MySQL, IBM DB2, Google BigQuery e SQLite.

Volumetrie e Ordini di Grandezza delle Basi di Dati. Quando diciamo che i database sono “grandi”, intendiamo volumi di dati tali per cui caricare tutto il database in RAM sarebbe impossibile o insostenibile dal punto di vista dei costi.

Ordini di Grandezza nelle Basi di Dati

Tipologia di Base di Dati e Utilizzo	Dimensione Tipica
Database Operativi Quotidiani (OLTP - acquisti, pagamenti)	1 – 5 Terabyte
Database per Analisi e Report (OLAP / Data Warehouse)	30 – 50 Terabyte
Database Scientifici (meteo, fisica delle particelle)	500 – 800 Terabyte
Numero Complessivo di Record	> 100 miliardi di righe

Nota: Per archiviare queste moli di dati, l'unico vero limite considerato dal DBMS è lo spazio fisico dei dischi (SSD o magnetici) e dello storage collegato in rete.

Condivisione, Ridondanza e Inconsistenza. In un'organizzazione complessa (es. un ateneo), uffici diversi (segreteria studenti, biblioteca, mensa) lavorano spesso sulle stesse informazioni (es. l'anagrafica degli studenti). Se ogni ufficio usasse un proprio archivio separato, nascerebbero grossi problemi. La frammentazione dei dati porta a due rischi tipici:

- **Ridondanza:** lo stesso dato viene duplicato in più archivi differenti (es. l'indirizzo di casa dello studente è salvato sia nel file della segreteria sia in quello della biblioteca).
- **Inconsistenza:** le copie duplicate non coincidono tra loro.
Esempio: Lo studente cambia residenza e comunica il nuovo indirizzo solo alla segreteria; la biblioteca mantiene il vecchio indirizzo. Il sistema ora contiene due informazioni reciprocamente contraddittorie sullo stesso fatto.

Centralizzare i dati in un unico database gestito da un DBMS evita la ridondanza inutile e impedisce l'inconsistenza.

Evoluzione Storica dell'Architettura per la Gestione dei Dati. Il modo di archiviare i dati si è trasformato radicalmente nel corso dei decenni:

- **Anni '70 (Senza DBMS):** i programmi leggevano e scrivevano direttamente su file di testo o file binari tramite il sistema operativo. Non c'era separazione tra il codice e il formato del file: se il file cambiava struttura, tutti i programmi andavano riscritti.
- **Anni '80 (Primi DBMS Relazionali):** nasce il DBMS come strato intermedio. I programmi (scritti ad esempio in C o Pascal) non toccano più i file su disco, ma chiedono al DBMS le tabelle e i dati di cui hanno bisogno.
- **Anni '90 (Logica Condivisa e DBMS Attivi):** parte delle regole di controllo viene spostata dentro il database con le **stored procedure** e i **trigger** (il database reagisce in automatico a modifiche e cancellazioni).
- **Anni 2000 (Architetture Web a Tre Livelli):** separazione netta tra interfaccia e dati:

Browser (Client) $\longleftrightarrow$ Server Applicativo (Java, PHP, Python) $\longleftrightarrow$ DBMS

- **Anni 2010 (Mobile e Cloud):** decine di dispositivi diversi (app iOS, Android, web) si collegano ad API centralizzate che dialogano con basi di dati distribuite e scalabili.

Evoluzione della Logica Procedurale: Stored Procedure vs Trigger

Negli anni '90 i DBMS hanno iniziato a eseguire logica direttamente sul server:

- **Stored Procedure:** blocchi di codice salvati direttamente sul database ed eseguiti su richiesta delle applicazioni; riducono il traffico di rete ma spesso non sono facilmente portabili tra DBMS di marche diverse.
- **Trigger (DBMS Attivi):** regole automatiche che scattano da sole secondo lo schema **Evento-Condizione-Azione**.
Esempio: Quando viene inserito un nuovo ordine (Evento), se la quantità ordinata supera le scorte in magazzino (Condizione), allora blocca l'inserimento e invia una notifica (Azione).

Garanzie del DBMS: Riservatezza, Affidabilità e Transazioni. Poiché la base di dati è condivisa da molte persone, il DBMS applica regole di accesso granulari:

- *Esempio A:* Uno studente può visualizzare tutti i propri voti, ma ha il permesso di modificare soltanto la propria password e il proprio recapito telefonico.
- *Esempio B:* Il docente del corso può inserire e modificare i voti dei propri studenti, ma non può visualizzare o alterare i dati fiscali o bancari degli stessi.

Se salta la corrente o il computer va in crash mentre stiamo salvando un dato, il database non deve corrompersi né lasciare operazioni a metà. Per garantire questa affidabilità, il DBMS utilizza il meccanismo delle **transazioni**.

La Transazione: Proprietà Fondamentali (Prospettiva di Sistema)

Una **transazione** è un blocco di operazioni che il sistema tratta come un'unica unità di lavoro indivisibile, garantendo tre caratteristiche principali:

1. **Atomicità (Tutto o Niente):** o tutte le istruzioni del blocco vanno a buon fine, oppure, al minimo errore o interruzione, il sistema annulla tutto e torna allo stato iniziale.
Esempio del bonifico: Se trasferiamo 100€ dal conto A al conto B, il sistema deve togliere 100€ da A e aggiungere 100€ su B. Se la macchina va in crash subito dopo aver tolto i soldi da A, la transazione viene annullata e i 100€ ritornano su A.
2. **Coerenza e Gestione della Concorrenza:** più utenti possono operare contemporaneamente sugli stessi dati senza calpestarsi i piedi.
Esempio del posto sul treno: Se due persone tentano di prenotare l'ultimo posto disponibile nello stesso identico secondo, il DBMS assegna il posto a una sola delle due e notifica l'esaurimento all'altra, evitando il sovrapprezzo o l'*overbooking*.
3. **Permanenza (Durabilità):** una volta che l'operazione è confermata (*commit*), le modifiche sono scritte sul disco (e registrate in un file di log protetto). Anche se il computer si spegne improvvisamente un istante dopo, il dato salvato non andrà perso.

Il DBMS deve raggiungere due traguardi pratici:

- **Efficienza:** velocità di esecuzione e basso consumo di memoria, grazie a indici intelligenti (es. alberi B-Tree) e all'ottimizzatore di interrogazioni.
- **Efficacia:** fornire linguaggi semplici e potenti (come SQL) che permettano a programmatori e analisti di ottenere le risposte cercate senza dover scrivere algoritmi a basso livello.

Limiti del File System Tradizionale. Un normale sistema operativo permette di salvare dati organizzandoli in file e cartelle (**file system**).

Il limite principale dei file system è la **grana grossa**: un file va generalmente aperto, letto e riscritto per intero. Se abbiamo un file con 2 milioni di righe e vogliamo modificare il numero di telefono di una sola persona, il programma applicativo deve rileggere il file, localizzare la riga corretta in memoria e riscrivere il file sul disco. Il DBMS, al contrario, lavora a **grana fine**: individua e modifica direttamente il singolo record in frazioni di secondo.

Descrizione dei Dati e Catalogo Centralizzato. La differenza più profonda tra i due modelli sta nel modo in cui è descritta la forma dei dati:

- **Nei sistemi con File System:** ogni singolo programma deve sapere a memoria come è formattato il file (es. "i primi 20 byte contengono il cognome, i successivi 10 la data"). Se modifichiamo il formato del file aggiungendo un campo, tutti i programmi esistenti smettono di funzionare finché non vengono riscritti.
- **Nei DBMS:** la struttura dei dati è definita in un solo posto centrale chiamato **catalogo** (o **dizionario dei dati**). I programmi non devono conoscere i dettagli fisici di memorizzazione: chiedono semplicemente le informazioni per nome al DBMS.

1.4 Modelli dei Dati, Architettura e Indipendenza

Il Modello dei Dati. Un **modello dei dati** è una collezione di concetti usati per organizzare e descrivere i dati.

Ogni modello offre specifici **costruttori di tipo**, simili a quelli usati nella programmazione (come array o struct). Nel **modello relazionale**, il costruttore principale è la **relazione** (comunemente vista come una **tabella**), composta da righe omogenee (tuple).

Schema e Istanza. In qualunque base di dati si distingue nettamente tra forma e contenuto:

- **Schema (Aspetto Intensionale):** l'intestazione, la struttura, i tipi di dato e i vincoli delle tabelle. È fisso e quasi mai soggetto a modifiche nel tempo.
- **Istanza (Aspetto Estensionale):** i dati concreti memorizzati nelle righe in un determinato momento. È molto dinamica, poiché righe vengono inserite, aggiornate o cancellate di continuo.

Esempio: Schema e Istanza nella Tabella Orario Lezioni

Si consideri una semplice tabella che organizza l'orario delle lezioni:

Schema della Relazione (Intestazione fissa, definita una volta sola)			
Corso (<i>Course</i>)	Docente (<i>Lecturer</i>)	Aula (<i>Room</i>)	Ora (<i>Time</i>)
Istanza della Relazione (Le righe di dati salvate in questo momento)			
Analisi I	Luigi Neri	N1	8:00
Basi di Dati	Pier Rossi	N2	9:45
Chimica	Nicola Mori	N1	9:45
Fisica I	Mario Bruni	N1	11:45
Fisica II	Mario Bruni	N3	9:45
Sistemi Informatici	Pier Rossi	N3	8:00

Le intestazioni definiscono **cosa** può contenere la tabella (lo *schema*); le sei righe di valori rappresentano il contenuto effettivo attuale (l'*istanza*).

Modelli Logici vs Modelli Concettuali. Esistono due grandi famiglie di modelli usate in momenti diversi della progettazione:

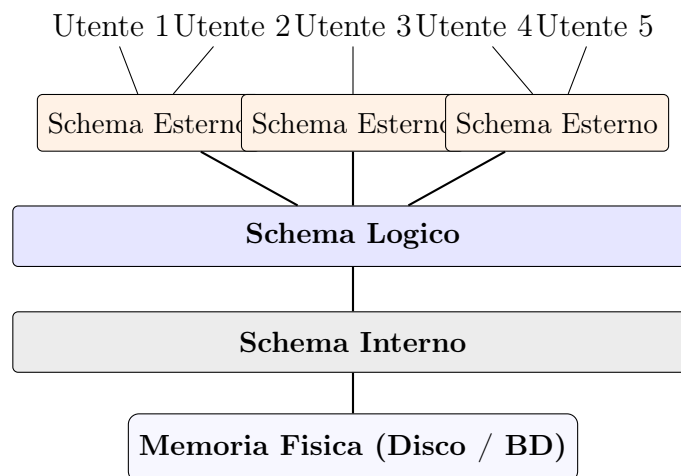
- **Modelli Logici:** sono i modelli compresi e usati dai DBMS per organizzare i dati su disco. Sono indipendenti dai dettagli fisici del disco, ma già vicini all'implementazione software.
Esempi: il modello **relazionale** (a tabelle), il modello gerarchico, a oggetti o documentale/-JSON (NoSQL).
- **Modelli Concettuali:** sono formalismi grafici e intuitivi usati per dialogare con i commitenti e descrivere la realtà senza pensare a quale software verrà usato in futuro.
Il più celebre: il **Modello Entità-Relazione (E-R)**.

Architettura dei DBMS: Livelli e Astrazione. In prima approssimazione, un sistema di gestione dati separa due aspetti (Architettura Semplificata a Due Livelli):

- **Schema Logico:** la vista complessiva della struttura (quali tabelle esistono, quali colonne contengono, quali relazioni ci sono tra loro).
- **Schema Interno (o Fisico):** come quei record sono disposti sui blocchi del disco, quali puntatori e quali file fisici vengono usati.

Architettura Standard ANSI/SPARC a Tre Livelli. Articola la visione dei dati su tre livelli:

1. **Livello Esterno (Schemi Esterni / Viste):** schermate e finestre ritagliate su misura per utenti specifici. Ogni gruppo di utenti vede solo i dati di proprio interesse.
2. **Livello Logico (Schema Logico):** la descrizione completa, unica e priva di ridondanze dell'intera base di dati.
3. **Livello Interno (Schema Interno):** l'organizzazione fisica su disco, i file di memoria e le strutture di indicizzazione.



Il Meccanismo delle Viste. Una **vista** è una tabella virtuale: non memorizza dati duplicati su disco, ma calcola le righe al momento eseguendo una query sulle tabelle reali sottostanti.

Esempio di Costruzione di una Vista: Accoppiamento Corsi-Aule

Consideriamo due tabelle di base memorizzate nel database:

Tabella Lezioni			Tabella Aule		
Corso	Docente	Aula	Nome	Edificio	Piano
Basi di Dati	Rossi	DS3	DS1	OMI	Terra
Sistemi	Neri	N3	N3	OMI	Terra
Reti	Bruni	N3	G	Pincherle	Primo
Controlli	Bruni	G			

Se agli studenti interessa solo sapere dove andare a lezione (senza vedere il nome del docente o le aule vuote), il DBMS può creare al volo la vista **AuleLezioni**, collegando le due tabelle tramite l'aula:

Corso	Aula	Edificio	Piano
Sistemi	N3	OMI	Terra
Reti	N3	OMI	Terra
Controlli	G	Pincherle	Primo

In questo modo l'utente vede una tabella pulita e pronta all'uso, senza che i dati siano stati duplicati sul disco.

Le Due Forme di Indipendenza dei Dati. Separare i tre livelli garantisce il principio di indipendenza dei dati:

- **Indipendenza Fisica:** possiamo cambiare il modo in cui i file sono scritti su disco, spostare il database su un SSD più veloce o aggiungere un indice per velocizzare le ricerche, **senza dover cambiare una sola riga** dei programmi applicativi o delle query SQL.
- **Indipendenza Logica:** possiamo aggiungere nuove tabelle o nuove colonne al database **senza rompere** i programmi preesistenti, che continuano a funzionare interagendo con le loro viste abituali.

1.5 Linguaggi per Basi di Dati e Modalità di Utilizzo

Classificazione Funzionale: DDL vs DML. Nei database si usano due categorie distinte di comandi:

- **DDL (Data Definition Language):** comandi per creare o modificare la struttura delle tabelle (lo schema).
Esempi: CREATE TABLE, ALTER TABLE, DROP TABLE.
- **DML (Data Manipulation Language):** comandi per inserire, cercare, modificare ed eliminare i record effettivi (l'istanza).
Esempi: SELECT, INSERT, UPDATE, DELETE.

```

1 CREATE TABLE orario (
2     corso    CHAR(20),
3     docente  CHAR(20),
4     aula     CHAR(4),
5     ora      CHAR(5)
6 );

```

Listing 1: Esempio DDL: creazione dello schema di una tabella

I Quattro Paradigmi di Interazione con la Base di Dati. Un utente o un programma può dialogare con il DBMS in quattro modi:

1. Linguaggio Interattivo Testuale (SQL a riga di comando). L'utente scrive direttamente comandi SQL in una console per ottenere subito i risultati a video:

```

1 SELECT Corso, Aula, Piano
2 FROM Aule, Lezioni
3 WHERE Nome = Aula
4     AND Piano = 'Terra';

```

Listing 2: Query SQL interattiva

2. SQL Immerso in un Linguaggio Ospite (*Embedded SQL*). I comandi SQL vengono inseriti all'interno di un linguaggio tradizionale (es. C o Pascal). Poiché SQL restituisce insiemi di righe mentre le variabili dei linguaggi procedurali leggono un valore alla volta, si usa un **cursore** per scorrere i risultati riga per riga:

```

1 write('Nome della città? '); readln(città);
2 EXEC SQL DECLARE P CURSOR FOR
3     SELECT NAME, INCOME
4     FROM PEOPLE
5     WHERE CITY = :città;
6 EXEC SQL OPEN P;
7 EXEC SQL FETCH P INTO :name, :income;
8 while SQLCODE = 0 do begin
9     write('Nome persona: ', name, ' Aumento? ');
10    readln(aumento);
11    EXEC SQL UPDATE PEOPLE
12        SET INCOME = INCOME + :aumento
13        WHERE CURRENT OF P;
14    EXEC SQL FETCH P INTO :name, :income;
15 end;
16 EXEC SQL CLOSE CURSOR P;

```

Listing 3: Esempio di Embedded SQL in Pascal con cursore

3. Linguaggi Procedurali Integrati nel DBMS (es. Oracle PL/SQL). Si tratta di linguaggi che uniscono istruzioni SQL a costrutti classici di programmazione (IF, cicli WHILE, gestione degli errori):

```

1 DECLARE
2     Stip NUMBER;
3 BEGIN
4     SELECT SALARY INTO Stip FROM EMPLOYEE
5     WHERE NUMBER = '575488' FOR UPDATE OF SALARY;
6
7     IF Stip > 30 THEN
8         UPDATE EMPLOYEE SET SALARY = SALARY * 1.1
9         WHERE NUMBER = '575488';
10    ELSE
11        UPDATE EMPLOYEE SET SALARY = SALARY * 1.15
12        WHERE NUMBER = '575488';
13    END IF;
14
15    COMMIT;
16 EXCEPTION
17    WHEN NO_DATA_FOUND THEN
18        INSERT INTO ERRORS

```

```
19      VALUES ( 'MATRICOLA NON ESISTENTE' , SYSDATE ) ;  
20 END ;
```

Listing 4: Esempio di blocco procedurale PL/SQL

4. Interfacce Grafiche (GUI e Maschere a Schermo). Gli utenti non tecnici (es. operatori di sportello, cassieri) non scrivono comandi SQL: usano maschere grafiche con campi da compilare, pulsanti e menu a tendina (es. Microsoft Access o pagine web gestionali). Dietro le quinte, il software genera ed esegue i comandi SQL corrispondenti.

1.6 Attori, Ruoli, Transazioni e Valutazione dei DBMS

I Quattro Gruppi di Ruoli Operativi. Nel mondo delle basi di dati lavorano quattro figure principali:

1. **Costruttori del DBMS:** gli ingegneri informatici che scrivono il motore del DBMS (progettano gli ottimizzatori di query, la gestione della memoria su disco e i compilatori interni).
2. **Progettisti del Database e DBA:** chi decide come strutturare le tabelle per uno specifico problema e ne amministra il funzionamento quotidiano.
3. **Sviluppatori Applicativi:** i programmatori che scrivono i programmi e le interfacce utente per interagire con i dati.
4. **Utenti:**
 - *Utenti Finali (Operativi):* usano interfacce guidate e maschere predefinite per compiti di routine (es. l'impiegato postale che registra un pacco).
 - *Utenti Casuali (Casual Users):* analisti o manager che interrogano il database formulando query SQL personalizzate per prendere decisioni strategiche.

L'Amministratore della Base di Dati (DBA). Il DBA (*Database Administrator*) è il custode e responsabile dell'efficienza e della salute del database. I suoi compiti principali sono:

- Monitorare le prestazioni del sistema e configurare gli indici per non rallentare le ricerche.
- Pianificare i salvataggi periodici (**backup**) e le procedure di ripristino dei dati in caso di guasti o disastri (*disaster recovery*).
- Gestire gli account, le password e le autorizzazioni di sicurezza di tutti gli utenti.
- Spesso collabora anche alla progettazione iniziale e all'evoluzione delle tabelle.

I Due Punti di Vista sulla Transazione. La parola “transazione” ha due sfumature:

- **Per l'utente finale:** è un'azione applicativa compiuta con uno scopo chiaro (es. fare un bonifico, comprare un biglietto aereo, iscriversi a un esame).
- **Per il DBMS:** è una serie indivisibile di comandi che devono essere eseguiti con successo al 100% oppure annullati del tutto, salvaguardando la coerenza dell'archivio anche in caso di errori o accessi concorrenti.

I Vantaggi (Pros) nell'Adozione di un DBMS. Scegliere un DBMS porta chiari vantaggi:

- **Dati Condivisi:** un'unica fonte di verità centralizzata per tutti i reparti dell'azienda.
- **Niente Duplicazioni Inutili:** si eliminano i rischi di avere dati disallineati e contraddittori.
- **Servizi Integrati:** il DBMS gestisce da solo sicurezza, controllo della concorrenza, transazioni e salvataggi.
- **Indipendenza dei Dati:** se si modifica l'hardware o la disposizione fisica dei file, non serve riscrivere i software applicativi.

Gli Svantaggi e i Costi (Cons). Un DBMS non è sempre la soluzione ideale in qualunque scenario, a causa di due fattori:

- **Costi Economici e Complessità:** le licenze enterprise sono onerose, richiedono server potenti e personale altamente qualificato (i DBA) per l'amministrazione.
- **Struttura Impegnativa (*Overhead*):** le funzioni di controllo transazionale, sicurezza e logging sono sempre attive e consumano risorse. Per compiti minuscoli o file di log elementari, un semplice file di testo gestito dal sistema operativo può risultare più veloce ed economico rispetto a un intero DBMS.